

AUTOMATION CONTROL MANUAL

ANSIBLE

PRODUCTION ANSIBLE

Secure, Tested and Delivered — **Vol 3**

Full chapter detail — rendered from content Markdown

VOL 1 – FOUNDATIONS

VOL 2 – ENGINEERING

VOL 3 – PRODUCTION
this book

TABLE OF CONTENTS

10	Security and Vault	03	12	CI/CD and Production	14
11	Testing and Quality	10	13	Advanced Automation	21
			A	Cheatsheet	25

[HOW TO READ]

Chapters render the full content Markdown: secrets, quality gates, delivery, recovery, labs, interview checks.

Version anchor: Same pins as Vol 1 (core 2.16+, lint 6+, molecule 6+); vault secrets via prompt, file or CI..

Production loop – every push



VAULT • BECOME • SECRETS

▶ WHAT – Least privilege + encrypted secrets

become escalates privilege (typically sudo) per play, role, or task. Ansible Vault encrypts variables and files so secrets live in Git safely and decrypt only at run time.

SECRETS STAY ENCRYPTED UNTIL RUN TIME

PLAINTEXT VAR



VAULT ENCRYPT



COMMIT SAFE



DECRYPT AT RUN

```
- name: Configure web fleet
  hosts: web
  become: true
  become_user: root
  vars_files:
    - group_vars/prod/vault.yml
  tasks:
    - name: Deploy TLS key
      ansible.builtin.copy:
        src: tls.key
        dest: /etc/nginx/tls.key
        mode: '0600'
```

```
ansible-vault encrypt group_vars/prod/vault.yml
ansible-playbook -i inventory.ini site.yml --ask-vault-pass
```

Vault file holds `vault_tls_key`, `vault_db_password` — referenced as `{{ vault_db_password }}`, never in plain vars.

▶ WHY – Automation is a privileged secret-spreader

Playbooks run as root across fleets and touch credentials. Plain-text secrets in Git leak to every clone; blanket `become: true` with passwordless sudo everywhere widens blast radius. Vault + scoped become give two guarantees: secrets encrypted at rest in version control, privilege granted narrowly per task.

VAULT · BECOME · SECRETS

▶ WHERE DOES IT FIT – Secure after organize

Roles/collections (Ch 09) define vars → Vault encrypts the sensitive subset → become scopes who applies them
→ testing (Ch 11) verifies without leaking → CI/CD (Ch 12) injects vault password from CI secrets

Do not confuse Ansible Vault (file/var encryption in-repo) with HashiCorp Vault (external secret store via lookup plugins). They compose: external vault supplies at run, Ansible Vault encrypts what must live in Git.

▶ HOW IT WORKS – Encrypt, scope, inject

Vault: `ansible-vault encrypt|decrypt|view|edit|rekey` per file; `encrypt_string` for inline values. Multiple vault IDs (`--vault-id prod@prompt`) separate environments. At run, password comes from prompt, file (`--vault-password-file`), or CI secret → file. `no_log: true` on secret-touching tasks suppresses output.

Become: `become: true + become_user + become_method: sudo` (or `su`, `runas` on Windows). Scope narrowly: play default `become: false`, escalate only tasks needing root. `--ask-become-pass` for interactive sudo password; in CI use `NOPASSWD` scoped sudoers or ephemeral privilege.

```
- name: Read app status as app user
  ansible.builtin.command: systemctl --user status app
  become: true
  become_user: app
  register: app_status
  changed_when: false
  no_log: false
```

VAULT • BECOME • SECRETS

▶ SUDO PITFALLS – Privilege side: sudoers and become identity

Most Ansible failures on Linux are not Ansible bugs — they are privilege, ownership, or interpreter mismatches. Learn the five usual suspects.

1. Sudoers scope. Grant the automation user exactly what plays need, nothing more:

```
ansible ALL=(ALL) NOPASSWD: /usr/bin/systemctl *, /usr/bin/apt-get *, /usr/bin/yum *
```

Blanket `ALL=(ALL) NOPASSWD:` ALL works in labs and horrifies auditors; passworded sudo needs `--ask-become-pass` (interactive) or CI-managed credentials. Old RHEL/CentOS images also enforce `Defaults requiretty` — sudo over non-interactive SSH fails until that line is removed for the automation user.

2. Become identity. `become_user` decides *who you become*, default root. Database tasks often need the service account instead:

```
- name: Vacuum app database
  community.postgresql.postgresql_query:
    query: VACUUM ANALYZE
    become: true
    become_user: postgres
```

VAULT • BECOME • SECRETS

▶ SUDO PITFALLS – Host side: ownership, SELinux, interpreter

3. Ownership and modes. Files created by a root-run task belong to root. Serve them to an app user and reads fail:

```
- name: Deploy app config
  ansible.builtin.template:
    src: app.conf.j2
    dest: /etc/app/app.conf
    owner: app
    group: app
    mode: '0640'
```

Always set owner/group/mode on security-relevant files; verify with a second task using `ansible.builtin.stat` when the service is picky.

4. SELinux contexts. On enforcing systems, correct mode with wrong context still denies:

```
ansible web -i inventory.ini -m ansible.builtin.command -a "getenforce" # Enforcing?
ansible web -i inventory.ini -m ansible.builtin.command -a "ls -Z /etc/app/"
```

Fix with `community.general.sefcontext + restorecon`, or `ansible.posix.selinux` booleans (`httpd_can_network_connect` is the classic) — never blanket `setenforce 0` in production.

5. Python interpreter. Modules execute in Python on the target. Minimal images lack it, or ship two versions:

```
[web:vars]
ansible_python_interpreter=/usr/bin/python3
```

Symptom `MODULE FAILURE: /bin/sh: python: command not found` → install `python3` via a raw bootstrap task (`ansible.builtin.raw: apt-get install -y python3`) before any module that needs Python. CRLF-edited playbooks from Windows editors also break YAML parsing — keep LF endings (see WSL setup, Ch 02).

VAULT · BECOME · SECRETS

▶ PRACTICAL – Encrypt and run without leaking

COMMAND

```
ansible-vault encrypt_string 's3cr3t-db-pass' --name 'vault_db_password'
```

Expected: vault_db_password: !vault | \$ANSIBLE_VAULT;1.1;AES256... — paste into group_vars/prod/vault.yml, commit safely.

COMMAND

```
ansible-playbook -i inventory.ini site.yml --ask-vault-pass --check --diff
```

Why it matters: proves encrypted vars decrypt at run and dry-run shows structure without exposing values (with no_log: true). Common failure: Attempting to decrypt but no vault secrets found → password file path wrong or --vault-id label mismatch.

```
ansible-vault view group_vars/prod/vault.yml      # read without decrypting to disk
ansible-vault edit group_vars/prod/vault.yml      # decrypt, edit, re-encrypt in one step
ansible-vault rekey group_vars/prod/vault.yml     # rotate the password on existing files
```

view for inspection, edit for changes, rekey for rotation — the file stays encrypted at every step except inside the editor.

VAULT · BECOME · SECRETS

▶ WHAT CAN FAIL

SYMPTOM	LIKELY CAUSE	PREVENTION
no vault secrets found	Missing --ask-vault-pass / wrong password file	CI secret → file → --vault-password-file; test locally with prompt first
Failed to escalate privilege	sudoers denies or password needed	Scoped NOPASSWD for automation user; --ask-become-pass interactively
Secret in logs	Task echoes var without no_log	no_log: true on secret tasks; audit with --check -vv in safe env
Vault + HashiCorp Vault confused	Same word, different tools	Ansible Vault = encrypted files in repo; HCP Vault = lookup at run (community.hashi_vault)

[GOTCHA]

Encrypting a file does not fix over-broad privilege. Vault protects secrets at rest; become scoping protects hosts at run. Do both — one without the other leaves half the attack surface open.

▶ REAL-WORLD CONNECTION

TLS key rotation: operator runs `ansible-vault rekey` per environment file, commits re-encrypted vault, opens PR showing only `!vault` blobs changed. CI holds vault password as protected secret, runs `--check --diff` for approval, then applies with `serial: 1`. Audit trail = Git history of who changed which encrypted file + CI run that decrypted it.

VAULT · BECOME · SECRETS

▶ LAB – Observe / Execute / Verify

Objective: secret committed encrypted, applied with scoped privilege. Prereqs: `group_vars/prod/vault.yml` with one test secret.

1. `ansible-vault encrypt group_vars/prod/vault.yml` → Observe blob, `git diff` shows no plaintext
2. `ansible-vault view group_vars/prod/vault.yml` → Verify round-trip decrypt
3. Run play with `--ask-vault-pass --check --diff` → Verify task references `{{ vault_db_password }}` without printing it (`no_log: true`)
4. Set `become: false` at play, `become: true` only on privileged task → Verify least-privilege scoping.

Failure drill: run without password → decrypt error → add `--ask-vault-pass` → converges.

▶ Interview check

Interview How do secrets reach hosts without living in plain text, and how is privilege scoped?

Secrets committed as Vault-encrypted vars/files, decrypted at run from prompt/file/CI secret with `no_log` guarding output. Privilege via `become` scoped per task (`become_user`, least-privilege sudoers), not blanket root everywhere.

[NEXT]

Secure automation must be trustworthy before it is triggered automatically. Next: lint, test, and Molecule quality gates.

LINT • CHECK • MOLECULE

▶ WHAT – Three quality gates

ansible-lint enforces style, FQCEN, and common mistakes statically. --check --diff dry-runs a playbook against real inventory without changing it. Molecule tests roles in ephemeral containers or VMs (create → converge → verify → destroy).



```

ansible-lint site.yml
ansible-playbook -i inventories/staging site.yml --check --diff
molecule test
  
```

Molecule cycle: create (ephemeral host) → converge (apply role) → verify (assert state) → destroy. Second converge should report ok with zero changed — the idempotence proof.

▶ WHY – Automation multiplies mistakes

A typo in one task runs on a hundred hosts in minutes. Lint catches the typo in seconds; check-mode previews blast radius; Molecule proves the role converges twice cleanly before CI ever touches staging. Without gates, every run is a fleet experiment.

▶ WHERE DOES IT FIT – Quality before triggers

```

Vault-secured roles (Ch 10) → lint (seconds) → check/diff (minutes, safe) → molecule (minutes,
ephemeral)
→ CI/CD (Ch 12) runs the same gates on every push
  
```

Local order mirrors CI order: fast static checks first, expensive ephemeral tests last. What passes locally must pass identically in CI — same lint version pinned, same molecule driver.

LINT • CHECK • MOLECULE

▶ HOW IT WORKS – Static, dry-run, ephemeral

Lint: rule-based (yaml, fqcn, name[prefix], no-changed-when). Pin `ansible-lint 6+` in dev requirements; `ansible-lint --fix` for safe auto-fixes. Check mode: modules report would-change without acting (`--check`); `--diff` shows file/service deltas. Command/shell tasks skip check unless `check_mode: true` is handled — another reason to prefer modules.

Molecule: `molecule init role acme.web -d docker`; `molecule.yml` defines platform, provisioner (`ansible_playbook`), verifier (`ansible asserts`). `verify.yml` asserts converged state:

```
- name: Verify
  hosts: all
  tasks:
    - name: Nginx is running
      ansible.builtin.assert:
        that:
          - "'nginx' in ansible_facts.services"
```

▶ PRACTICAL – Gate a role locally

COMMAND

```
ansible-lint roles/web/ && ansible-playbook -i inventories/staging site.yml --check --diff
```

Meaning: static rules, then dry-run delta. Expected: lint clean, `--diff` shows only intended file lines. Any unexpected changed line blocks the commit.

COMMAND

```
molecule test
```

Expected: create → converge (changed) → converge (ok, idempotent) → verify passed → destroy. Second-converge `changed=0` is the quality signature.

Why it matters: three gates catch three bug classes (style/FQCN, blast radius, convergence). Common failure: lint fqcn errors on short names → rewrite to `ansible.builtin.*`.

LINT • CHECK • MOLECULE

▶ WHAT CAN FAIL

SYMPTOM	LIKELY CAUSE	PREVENTION
fqcn[action-core] lint errors	Short module names (copy, template)	FQCN everywhere; <code>ansible-lint --fix</code> once, then enforce
<code>--check</code> shows nothing but real run changes	<code>command/shell</code> tasks skip check mode	Prefer modules; add <code>check_mode: true</code> handling or <code>changed_when</code>
Molecule verify fails	Role converged but assertion wrong	Assert observable state (service, file, port), not task output
Second converge still changed	Non-idempotent content (timestamp, random)	Remove volatile rendering; retest until <code>changed=0</code>

[GOTCHA]

A green molecule run whose second converge still reports CHANGED is a failed idempotence test wearing a passing disguise. Gate on `changed=0`, not just exit code.

▶ REAL-WORLD CONNECTION

PR pipeline: lint (30s) → check/diff against staging inventory (2min, artifact of `--diff`) → molecule (5min). Reviewer approves the `--diff` artifact, not a description. Merge to main triggers staging converge; production requires manual approval + same gates with prod inventory in check mode.

▶ LAB – Observe / Execute / Verify

Objective: red → green quality loop. Prereqs: roles/web from Ch 09, molecule + docker available.

1. Introduce short name (copy:) → `ansible-lint roles/web/` → Observe fqcn failure
2. Fix to `ansible.builtin.copy` → Verify lint clean. Explain: static gate.
3. Run `--check --diff` → Observe planned deltas. Explain: blast-radius preview.
4. `molecule test` → Verify second converge `changed=0` + verify passed. Explain: ephemeral convergence proof.

LINT • CHECK • MOLECULE

Failure drill: add `{{ ansible_date_time.iso8601 }}` to template → molecule second converge stays changed → remove volatile → green.

▶ Interview check

Interview What does each gate prove that the others do not?

Lint proves style/FQCN correctness statically. Check/diff proves blast radius against real inventory without changing it.

Molecule proves the role converges to testable state twice (idempotent) in a clean ephemeral host.

[NEXT]

Quality gates pass locally. Next: run them on every push and drive fleets — CI/CD and production.

CI/CD – COMMIT TO FLEET

▶ WHAT – Every push drives automation safely

CI/CD triggers ansible-playbook after Git push, with per-environment inventory. The pipeline runs the Ch 11 gates, previews with `--check --diff`, requires approval for prod, then converges the fleet and feeds results to observability.



```
Git push → CI (lint) → molecule → ansible-playbook -i inventories/staging --check --diff
→ approval → ansible-playbook -i inventories/prod → observability → feedback to Git
```

Project structure:

```
inventories/{staging,prod}/hosts.yml
group_vars/{all,staging,prod}.yaml
host_vars/
roles/
site.yml
ansible.cfg
requirements.yml
```

▶ WHY – Manual runs do not survive scale

SSH-loop runs depend on who runs them, from which laptop, with which vars. CI gives one runner, pinned versions, secret-injected vault passwords, and an audit trail per commit. Staging-then-prod with identical playbooks makes promotion a change of `-i` path, not a new procedure.

CI/CD – COMMIT TO FLEET

▶ WHERE DOES IT FIT – Production loop

Terraform provisions → Ansible configures (this series) → Docker packages → K8s orchestrates
→ CI triggers → observability verifies → feedback to Git

Ansible owns inside-OS convergence. Execution tuning (strategy, forks, serial) and inventory at scale (constructed inventory, group_vars layering) keep that convergence fast and scoped as fleets grow.

▶ HOW IT WORKS – Strategies, scale, pipeline

Execution strategies: linear (default, lockstep per task) vs free (hosts race ahead). forks: 20 caps parallelism; serial: 1 (or "25%") rolls hosts in batches for safe restarts. `ansible-inventory --graph` verifies selection before any run.

```
- name: Rolling web update
  hosts: web
  serial: 1
  tasks:
    - name: Render config
      ansible.builtin.template:
        src: nginx.conf.j2
        dest: /etc/nginx/nginx.conf
      notify: Restart nginx
  handlers:
    - name: Restart nginx
      ansible.builtin.service:
        name: nginx
        state: restarted
```

GitHub Actions sketch: checkout → setup Python → `pip install ansible ansible-lint molecule → ansible-galaxy install -r requirements.yml → lint → molecule test → check/diff staging (artifact) → manual approval → run prod` with vault password from CI secrets.

Inventory at scale: dynamic/construction plugins generate groups from cloud tags; group_vars layers defaults → env → host; `ansible-inventory -i inventories/prod --graph --vars` is the pre-flight check.

CI/CD – COMMIT TO FLEET

▶ PRACTICAL – Staging dry-run, prod rolling

COMMAND

```
ansible-inventory -i inventories/prod --graph && ansible-playbook -i inventories/prod site.yml --check --diff
```

Meaning: prove selection, then preview deltas. Expected: graph shows web/db counts; --diff lists only intended lines. Any surprise line blocks promotion.

Why it matters: the --diff artifact is what approvers review — not a prose description. Common failure: CI env lacks vault password → decrypt error → wire CI secret to --vault-password-file via temp file, never echo.

```
ansible-playbook -i inventories/prod site.yml --check --diff    # reviewed artifact
ansible-playbook -i inventories/staging site.yml               # converge staging first
```

WHAT HAPPENED: dry run against the prod inventory; zero changes made, full delta shown. **WHY IT MATTERS:** the diff artifact is what approvers review — not a prose description of intent. **COMMON FAILURE:** CI lacks the vault password → decrypt error; fix by wiring the secret to a file, never echoing it into logs.

CI/CD – COMMIT TO FLEET

► ISSUE CATALOG – Reachability to data (1–7)

Read top to bottom: connection first, then privilege, then data, then convergence. Nine runs out of ten die in rows 1–3.

#	SYMPTOM	LIKELY CAUSE	VERIFY	FIX
1	UNREACHABLE	IP wrong, host down, firewall, bastion missing	ping IP → ssh user@ip by hand → -m ping -vvv	Correct ansible_host, open port 22, set ProxyJump
2	Permission denied (publickey)	Key not deployed, wrong key, 0600 broken on /mnt/c	ssh -i key user@ip by hand	ssh-copy-id, point ansible_ssh_private_key_file, keep keys in WSL home
3	Failed to escalate privilege	Sudo needs password / not granted / requiretty	sudo -n true on target as automation user	Scoped NOPASSWD, --ask-become-pass, drop requiretty
4	MODULE FAILURE: python: command not found	No Python on minimal image	raw: which python3	Bootstrap python3 via raw, set ansible_python_interpreter
5	ERROR! Syntax Error (YAML)	Indent mix, tabs, CRLF from Windows editor	yamllint, cat -A shows ^M	2-space indent, LF endings, lint gate in CI
6	AnsibleUndefinedVariable	Typo, group_vars/ beside wrong file	ansible-inventory --graph --vars	Fix name/path; guard with default()
7	Wrong value, no error (precedence)	Higher-specificity source wins silently	debug: var=name per host	Move value to host_vars/ or override deliberately

CI/CD – COMMIT TO FLEET

► ISSUE CATALOG – Convergence and delivery (8–14)

Convergence and delivery failures behave differently: the transport works, the logic does not.

#	SYMPTOM	LIKELY CAUSE	VERIFY	FIX
8	template error while templating string	Undefined var or bad filter in Jinja	Render with debug first	default() guards, test filters in isolation
9	Handler never runs	Task OK (converged) or name/listen mismatch	Change a var to force CHANGED	Fix handler name, use listen: aliases
1 0	couldn't resolve module/action	Collection missing or short name	ansible-galaxy collection list	FQCN + requirements.yml install
1 1	no vault secrets found	Password file path / --vault-id label wrong	Run once with --ask-vault-pass	Wire CI secret to --vault-password-file
1 2	Task always CHANGED	command/shell without guard, volatile template content	Re-run: still changed?	Prefer modules; changed_when/creates; remove timestamps from templates
1 3	Fleet run crawls	linear strategy, forks default 5, fact gathering everywhere	Time with --verbose, count hosts	forks: 20+, strategy: free where safe, gather_facts: false when unused, serial for rolling
1 4	Works locally, fails in CI	Pinned versions differ, vault/SSH secrets absent in CI	Diff ansible --version + env both sides	Pin all versions, identical install steps, CI secret inventory

Prevention stack (cheap → expensive): ansible-lint on every save → --check --diff before every real run → staging converge before prod → serial + approval for prod → observability on changed counts after.

WHERE ANSIBLE SITS IN PRODUCTION



Terraform hands addresses; Ansible owns everything inside the machine.

CI/CD – COMMIT TO FLEET

► WHAT CAN FAIL – Troubleshooting matrix

SYMPTOM	VERIFY	FIX	PREVENTION
UNREACHABLE	<code>ansible web -m ping -vvv</code>	user/key/host, bastion, <code>ansible_ssh_common_args</code>	ping gate before converge in CI
YAML parse error	<code>yamllint</code> , line number	indent/colon/quotes	lint gate blocks merge
Precedence surprise	<code>ansible-inventory --graph --vars + debug</code>	correct <code>group_vars/</code> placement	staging check/diff review
Handler silent when restart needed	task OK or name mismatch	force change or fix listen	validate + <code>--diff</code> inspection
Vault decrypt fail in CI	password file present?	CI secret → file → <code>--vault-password-file</code>	staging pipeline runs first
Env mismatch (works locally, fails CI)	pinned versions?	pin <code>ansible</code> , <code>lint</code> , collections in <code>requirements*.yaml</code>	identical local/CI install steps

► REAL-WORLD CONNECTION

End-to-end: Terraform applies infra → outputs inventory → CI runs Ansible staging converge → `--diff` approved → prod rolling converge (`serial: "25%"`) → Prometheus changed-count drop and service health confirm → Grafana dashboard links back to commit SHA. Drift next week re-runs the same commit — CHANGED hosts are the drift report.

► LAB – Observe / Execute / Verify

Objective: promote same commit staging → prod. Prereqs: `inventories/staging`, `inventories/prod`.

- `ansible-inventory -i inventories/staging --graph` → Observe selection
- `--check --diff staging` → Execute, capture artifact → Verify intended deltas only
- Converge staging → Verify changed then re-run `changed=0`
- Repeat check/diff against prod → approval → rolling converge `serial: 1`.

CI/CD – COMMIT TO FLEET

Failure drill: point prod run at staging inventory → graph mismatch → abort via selection check before converge.

▶ Interview check

Interview How does a commit safely reach production fleet?

Push runs lint → molecule → staging check/diff (reviewed artifact) → staging converge → prod check/diff → approval → rolling prod converge with vault secrets from CI, verified by inventory graph and observability — same playbook, different - i path.

[NEXT]

Fleet delivery works. Next: the orchestration edge — delegation, async, blocks, and running subsets.

DELEGATION • ASYNC • BLOCKS • TAGS

▶ WHAT – Orchestration controls

`delegate_to` runs a task on a different host than the play target. `local_action` runs on the control node. `async/poll` runs long tasks without blocking. `block/rescue/always` groups tasks with recovery. `tags` selects subsets.

RUN ELSEWHERE, RUN LONG, RUN SAFE

DELEGATE_TO
act on another host

ASYNC + POLL
long jobs

BLOCK / RESCUE
recover

TAGS + LIMIT
subsets

```
- name: Migrate with recovery
  block:
    - name: Run migration
      ansible.builtin.command: /opt/migrate.sh
      async: 600
      poll: 10
  rescue:
    - name: Roll back
      ansible.builtin.debug:
        msg: "migrate failed, rolling back"
  always:
    - name: Ensure app started
      ansible.builtin.service:
        name: app
        state: started

- name: Drain from load balancer
  ansible.builtin.uri:
    url: "http://lb/drain?host={{ inventory_hostname }}"
  delegate_to: localhost
  run_once: true
```

Run subsets: `ansible-playbook site.yml --tags deploy --limit web`.

DELEGATION • ASYNC • BLOCKS • TAGS

▶ WHY – Real runs need escape hatches

Migrations fail mid-fleet; long jobs exceed SSH timeouts; load balancers must drain one host at a time; operators need to run only deploy tags at 2am. Without these controls every edge case becomes a separate playbook — with them, one playbook handles happy path, slow path, and failure path.

▶ WHERE DOES IT FIT – Edge after production

CI/CD delivery (Ch 12) runs the standard path → this chapter handles the non-standard:
different host (delegate) → long job (async) → failure (block/rescue) → subset (tags/limit)

These are execution modifiers, not new declarative models. Teach them last because each assumes the learner already reads plays, data flow, handlers, and rolling updates fluently.

▶ HOW IT WORKS – Where, how long, what if, which

Delegation: task executes on `delegate_to` host but `inventory_hostname` still refers to original target — use for LB/API calls about the target. `run_once: true + delegate_to: localhost` runs a single API call, not once per host. Async: `async: 600` (max seconds) + `poll: 10` (check interval); `poll: 0` fires and continues (collect with `async_status` later). Blocks: rescue runs only if block fails; always runs regardless — the try/except/finally of playbooks. Tags: `tags: [deploy, never]`; `--tags`, `--skip-tags`, `--list-tags`; never tags run only when requested.

```
- name: Long firmware job
  ansible.builtin.command: /opt/flash.sh
  async: 1800
  poll: 0
  register: flash_job

- name: Wait for firmware
  ansible.builtin.async_status:
    jid: "{{ flash_job.ansible_job_id }}"
  register: flash_result
  until: flash_result.finished
  retries: 60
  delay: 30
```

DELEGATION • ASYNC • BLOCKS • TAGS

▶ PRACTICAL – Prove rescue + async

COMMAND

```
ansible-playbook -i inventory.ini rescue-demo.yml --tags migrate
```

Expected: failing block task → rescue debug runs → always service start runs → play recap failed=0 (recovered) with rescue visible. Remove rescue → recap failed=1 — proves recovery changed the outcome.

Why it matters: rescue converts fleet-stopping failures into logged recoveries with service guarantees. Common failure: delegate_to without run_once on LB drain → N duplicate drains → add run_once: true.

▶ WHAT CAN FAIL

SYMPTOM	LIKELY CAUSE	PREVENTION
LB drained N times	delegate_to runs per host	run_once: true for singleton API calls
Async job lost	poll: 0 without async_status follow-up	until/retries/delay poller task with ansible_job_id
rescue masks real failure	Over-broad rescue hides root cause	Rescue logs + re-raises (fail:) when unrecoverable; alert, not silence
--tags runs nothing	Tag typo or never tag not requested	--list-tags before run; audit tag names in review
command always CHANGED in block	Imperative task without guard	changed_when, creates/removes, or failed_when: rc not in [0]

[GOTCHA]

rescue is not exception forgiveness — it is a second playbook branch. If rescue cannot actually restore service, fail loudly with ansible.builtin.fail so CI and observability see red, not a polite debug message.

▶ REAL-WORLD CONNECTION

Zero-downtime deploy: serial: 1 → delegate_to: localhost drains host from LB (run_once per batch item) → deploy + handler restart → uri health check with until/retries → re-add to LB in always. --tags rollback runs only the restore branch. Same playbook deploys, verifies, and recovers — the runbook and the automation are one file.

DELEGATION • ASYNC • BLOCKS • TAGS

▶ LAB – Observe / Execute / Verify

Objective: drain → deploy → verify → recover. Prereqs: rescue-demo.yml with block/rescue/always + drain task.

1. Run with `--list-tags` → Observe available subsets
2. Run `--tags migrate` with failing migration → Verify rescue + always both ran, recap recovered
3. Convert long task to async: `300 / poll: 0 + async_status poller` → Verify non-blocking completion
4. Add `delegate_to: localhost / run_once: true` to drain → Verify single drain for whole batch.

Failure drill: remove `run_once` → observe duplicate drains → restore; remove rescue → observe hard failure → restore.

▶ Interview check

Interview When do you delegate, async, block, or tag?

Delegate when the action concerns another host (LB, API, control node). Async when the job outlasts comfortable SSH blocking. Block/rescue/always when failure needs scoped recovery with guaranteed cleanup. Tags + limit when the operator must run a named subset safely.

[NEXT]

Series complete: foundations declare state, engineering varies and organizes it, production secures, tests, delivers, and recovers it. Return to Ch 01 decision tree for the next fleet.

CHEATSHEET – SECURE TO DELIVER

A Vault and Become

NEED	COMMAND
Encrypt file	<code>ansible-vault encrypt</code>
Inline secret	<code>encrypt_string --name</code>
Run	<code>--ask-vault-pass / password-file</code>
Scope root	<code>become: true per task</code>

B Gates and Delivery

NEED	COMMAND
Lint	<code>ansible-lint site.yml</code>
Preview	<code>--check --diff</code>
Ephemeral	<code>molecule test</code>
Subset	<code>--tags x --limit y</code>

● OK – converged, no change

● CHANGED – converged, did change

❌ FAILED – did not converge

END PRODUCTION · VOL 3

SERIES COMPLETE

Declare state, vary it per host, secure it, test it, deliver it — and recover.